

ASSIGNMENT REPORT 1: PROCESS AND THREAD IMPLEMENTATION

CENG 2034, OPERATING SYSTEMS

Berkay Dündar
berkaydundar@posta.mu.edu.tr

Thursday 24th April, 2025

Abstract

This report compares processes and threads in Python, focusing on their performance in I/O bound and CPU-bound tasks. It explores appropriate scenarios for each approach, analyzes performance, and provides recommendations based on the findings.scenarios.

1 Introduction

The aim of this report is to evaluate the implementation of processes based and thread based parallel programming in Python. The specific objectives include understanding the differences between processes and threads, analyzing their performance in I/O-bound and CPU bound tasks, and providing recommendations for their optimal use.

2 Assignments

2.1 Research and Theoretical Understanding

This part contains theoretical explanations and comparisons of specified topics.

2.1.1 Process and Threads

When computer programs are executed, the operating system allocates resources to them and manages their operation. These running programs are referred to as processes. Processes are independent units that operate separately and have their own memory space. Within each process, there can be one or more threads. A thread is a smaller unit of execution that runs within a process. Threads share the same memory space, which allows for faster data sharing. However, this also introduces potential issues such as the need for synchronization.

2.1.2 Python's Multiprocessing and Threading Modules

Python offers two primary modules for parallel programming: threading and multiprocessing. These modules are designed to address different types of workloads. Threading module allows multiple threads to run within a single process. Since threads share memory, they are lightweight and efficient for I/O-bound tasks where CPU usage is minimal, such as downloading files

or reading from disk. On the other hand, multiprocessing module creates separate processes for each task. Each process runs independently with its own memory space. This makes multiprocessing ideal for CPU-intensive tasks like matrix operations or calculating prime numbers.

2.1.3 Global Interpreter Lock (GIL)

The Global Interpreter Lock (GIL) is a mechanism used in the CPython implementation of Python that allows only one thread to execute Python bytecode at any given time. This can limit performance in CPU-bound programs because only one thread runs at a time, regardless of the number of cores. However in I/O-bound tasks, the GIL is less of a problem since threads spend most of their time waiting. For CPU-intensive operations, using multiprocessing is often a better choice as it bypasses the GIL and allows true parallelism.

2.1.4 Advantages and Disadvantages of Using Threads vs. Process

Using processes instead of threads in Python has both strengths and limitations, especially due to the Global Interpreter Lock (GIL). With threads, it's easier to share memory and manage communication but because of the GIL, threads don't actually run in parallel on multiple cores when doing CPU-bound tasks. This limits their performance in intensive computations. On the other hand, processes run in separate memory spaces and can truly execute in parallel, making them more suitable for CPU-bound workloads. However, this also means more overhead, as inter process communication is slower and consumes more resources compared to threads. So threads offer lighter, more efficient performance for I/O-bound tasks, while processes provide true concurrency at the cost of higher memory and communication overhead.

2.2 I/O-Bound Task

2.2.1 Threading for I/O Bound Task

In this implementation, the `threading` module is used to perform multiple file downloads in parallel. Each download task runs in a separate thread, which allows the program to stay responsive while waiting for data from the network. Since I/O-bound tasks usually involve waiting for external sources rather than using CPU power, threads are well-suited for this type of work. The program measures the total execution time and peak memory usage using `time` and `tracemalloc`. This setup helps demonstrate how threading can speed up I/O operations with minimal resource usage.

2.2.2 Multiprocessing for I/O Bound Task

This version uses the `multiprocessing` module to handle file downloads. Each download is performed in its own process, running independently from the others. While multiprocessing allows for real parallelism, especially for CPU-heavy tasks, it can add unnecessary overhead in I/O-bound scenarios. Processes are heavier to start and manage compared to threads. The goal of this implementation is to compare how multiprocessing performs in an I/O-heavy situation where threading is usually more efficient. Execution time and memory usage are again recorded for performance comparison.

2.3 CPU-Bound Task

2.3.1 Threading for CPU-Bound Task

The third scenario focuses on CPU-bound operations by calculating prime numbers within given ranges using multiple threads. Each thread runs a separate calculation. However, due to Python's Global Interpreter Lock (GIL), only one thread can execute Python code at a time, which reduces the effectiveness of threading for such tasks. This implementation is useful to observe the limitations of threads in CPU-intensive work. By collecting timing and memory data, the code shows that threads may not be the best choice for high-computation programs.

2.3.2 Multiprocessing for CPU-Bound Task

The final example also calculates prime numbers, but this time using multiprocessing. Each task is run in a separate process, allowing full parallel execution across CPU cores. This bypasses the GIL restriction and shows the real benefit of multiprocessing for CPU-bound problems. The implementation aims to highlight how using processes can significantly reduce execution time in computational workloads. Like the other cases, memory and time are monitored to get a clear view of the program's resource demands.

3 Results

Scenario	Approach	Execution Time (s)	Memory Usage (MB)
I/O-Bound	Threads	0.52 seconds	1.02 MB
I/O-Bound	Processes	1.54 seconds	0.34 MB
CPU-Bound	Threads	1.52 seconds	1.04 MB
CPU-Bound	Processes	0.35 seconds	0.62 MB

Table 1: Performance Metrics for I/O-Bound and CPU-Bound Tasks

The experiments show clear performance differences between threads and processes in different scenarios. For I/O tasks like file downloads, threads performed better (0.52s) compared to processes (1.54s) because they handle waiting operations more efficiently. However, threads used slightly more memory (1.02MB vs 0.34MB) due to their shared memory structure.

For CPU-intensive tasks, processes were significantly faster (0.35s vs 1.52s for threads) because they bypass Python's GIL limitation. While processes required more memory than threads (0.62MB vs 1.04MB), their ability to use multiple CPU cores made them much more effective for heavy computations.

These results confirm that threads work best for I/O operations, while processes are superior for CPU-heavy tasks. The choice depends on whether your program needs to wait for external operations or perform intensive calculations.

4 Conclusion

In conclusion, threads are generally more effective for I/O-bound tasks due to their quicker response time and lighter memory footprint. Since such tasks often involve waiting for external input or data, threading helps maintain responsiveness without much overhead. For CPU-bound tasks, processes offer better performance because they bypass the restrictions of the Global

Interpreter Lock (GIL), allowing true parallel execution. While threads struggle under heavy computation due to GIL, processes make better use of multi-core systems. Therefore, when choosing between threads and processes, the nature of the task and the impact of the GIL should be carefully considered. Selecting the right method can greatly improve both speed and resource usage.

References

- [1] Python Software Foundation. Python multiprocessing documentation. <https://docs.python.org/3/library/multiprocessing.html>.
- [2] Python Software Foundation. Python threading documentation. <https://docs.python.org/3/library/threading.html>.
- [3] IBM. Understanding threads and processes. <https://www.ibm.com/docs/en/aix/7.2?topic=threads-understanding-processes>.
- [4] Andrew S. Tanenbaum and Herbert Bos. *Modern Operating Systems*. Pearson International, 2014.
- [5] Wikipedia contributors. Global interpreter lock. https://en.wikipedia.org/wiki/Global_interpreter_lock.